Python Cheat Sheet

Deine Referenz für den Einstieg ✨

① Variablen

Benannte Speicherplätze für Daten. Python erkennt den Typ automatisch, keine Deklaration nötig.

- `x = 10` — Ganzzahl speichern
- `name = "Anna"` — Text speichern
- `aktiv = True` — Wahrheitswert
- `pi = 3.14` — Kommazahl
- Groß-/Kleinschreibung zählt: `x ≠ X`
- `a, b = 1, 2` — Mehrfachzuweisung
- `a, b = b, a` — Werte tauschen
- `x, y, z = [1, 2, 3]` — entpacken

💡 *Namen in snake_case: `mein_name` statt `meinName`.*

② Datentypen

Jede Variable hat einen Typ. Python wählt ihn automatisch, du kannst ihn abfragen oder umwandeln.

- `int` — Ganzzahl: `42`, `-7`, `0`
- `float` — Kommazahl: `3.14`, `-0.5`
- `str` — Text: `"Hallo"`, `'Hi'`
- `bool` — `True` oder `False`
- `None` — kein Wert, leer
- `type(x)` — Typ abfragen
- `isinstance(x, int)` — Typ prüfen
- `int("42")` / `str(10)` — umwandeln

💡 *`int("42")` → `42`, `float("3.14")` → `3.14`, `str(10)` → `"10"`.*

③ Strings

Zeichenketten in Anführungszeichen. f-Strings sind der modernste Weg um Variablen direkt einzufügen.

```
# f-String
name = "Anna"
print(f"Hallo {name}!")  # Hallo Anna!
print(f"{3.14:.2f}")     # 3.14

# String-Methoden
text = "  Hallo Welt  "
text.strip()              # "Hallo Welt"
text.upper()              # "  HALLO WELT  "
text.replace("Welt","Du") # "  Hallo Du  "
text.split(" ")           # ["", "", "Hallo"...]

# Slicing
s = "Python"
s[0]   # "P"
s[-1]  # "n"
s[1:4] # "yth"
```

💡 *Strings sind unveränderlich. Jede Methode gibt einen neuen String zurück.*

④ Listen

Speichern mehrere Werte geordnet. Index startet bei 0. Listen wachsen dynamisch.

```
# Erstellen und zugreifen
z = [1, 2, 3, 4, 5]
z[0]     # 1  — erstes Element
z[-1]    # 5  — letztes Element
z[1:4]   # [2, 3, 4] — Slice
z[::-1]  # [5,4,3,2,1] — umgekehrt

# Bearbeiten
z.append(6)     # am Ende anhängen
z.insert(0,99)  # an Position einfügen
z.remove(3)     # Wert entfernen
z.sort()        # sortieren (in-place)
len(z)          # Anzahl Elemente
3 in z          # True / False
```

💡 *`sorted(z)` gibt sortierte Kopie zurück. `list(range(5))` → `[0,1,2,3,4]`.*

Python Cheat Sheet

Deine Referenz für den Einstieg ✨

⑤ Bedingungen

Mit if/elif/else trifft dein Programm Entscheidungen. 4 Leerzeichen Einrückung ist Pflicht.

```
if x > 10:
    print("groß")
elif x == 10:
    print("genau 10")
else:
    print("klein")

# and / or / not
if x > 0 and x < 100:
    print("gültig")

# Kurzform (ternary)
s = "ja" if x > 10 else "nein"
```

⚠ `==` vergleicht, `=` weist zu, `is` prüft Identität.

⑥ Schleifen

for läuft über Sequenzen, while so lange eine Bedingung gilt. break bricht ab, continue überspringt.

```
# for mit range()
for i in range(5):       # 0,1,2,3,4
    print(i)

for i in range(2,10,2):  # 2,4,6,8
    print(i)

# Index + Wert
for i, v in enumerate(liste):
    print(i, v)

# while mit break
x = 10
while x > 0:
    if x == 5: break
    x -= 1
```

💡 `for _ in range(3):` wenn der Index nicht gebraucht wird, _ als Platzhalter nutzen.

⑦ Funktionen

Bündeln wiederverwendbaren Code. Default-Werte machen Parameter optional.

```
def begrüße(name, gruss="Hallo"):
    return f"{gruss}, {name}!"

print(begrüße("Anna"))
# Hallo, Anna!
print(begrüße("Tom", "Hi"))
# Hi, Tom!

# Mehrere Rückgabewerte
def min_max(lst):
    return min(lst), max(lst)

lo, hi = min_max([3, 1, 9])
# lo=1, hi=9
```

💡 Ohne `return` gibt die Funktion `None` zurück.

⑧ Dictionaries

Schlüssel-Wert-Paare, wie ein Wörterbuch. Ideal wenn Daten einen Namen statt einen Index haben sollen.

```
person = {
    "name": "Anna",
    "alter": 25
}

person["name"]          # "Anna"
person["job"] = "Dev"   # hinzufügen
del person["alter"]     # entfernen
person.get("x", 0)      # 0 statt KeyError
"name" in person        # True

# Über Keys & Values
for k, v in person.items():
    print(k, v)
```

💡 `.get(key, fallback)` verhindert KeyError. `.setdefault(key,[])` setzt nur wenn fehlt.

Python Cheat Sheet

Deine Referenz für den Einstieg ✨

Anfänger 3 / 3

⑨ Fehlerbehandlung

Exceptions können das Programm abstürzen lassen. try/except fängt sie ab, finally läuft immer.

```
try:
    zahl = int(input("Zahl: "))
    print(10 / zahl)
except ValueError:
    print("Keine Zahl!")
except ZeroDivisionError:
    print("Nicht durch 0!")
finally:
    print("Fertig.")

# Exception auslösen
raise ValueError("Negativ!")
```

💡 *Immer spezifische Exceptions fangen, nie nur `except:` allein.*

⑩ Operatoren

Führen Berechnungen, Vergleiche und logische Verknüpfungen durch. Python unterstützt verkettete Vergleiche.

- `+ - * /` — Grundrechnung
- `//` — Ganzzahldivision: `7//2 = 3`
- `%` — Modulo: `7%2 = 1`
- `**` — Potenz: `2**10 = 1024`
- `== ≠ < > ≤ ≥` — Vergleich
- `and or not` — Logisch
- `in not in` — Zugehörigkeit
- `is is not` — Identität
- `x += 1` kurz für `x = x + 1`
- `0 < x < 100` — verkettet

💡 *`==` vergleicht Werte, `is` prüft ob zwei Variablen auf dasselbe Objekt zeigen.*

⑪ Nützliche Built-ins

Immer verfügbar, kein Import nötig. Decken die häufigsten Aufgaben ab.

- `print(x, sep=", ")` — Ausgabe
- `input("Text")` — Benutzereingabe
- `len(x)` — Länge von Liste oder String
- `range(start, stop, step)`
- `type(x)` / `isinstance(x, int)`
- `int() float() str() bool()`
- `min() max() sum()` — auf Listen
- `sorted(x)` — sortierte Kopie
- `abs(x)` — Absolutwert
- `round(x, 2)` — runden

💡 *`enumerate()`, `zip()` und `map()` sind eingebaut und sehr nützlich.*

⑫ Module & Imports

Fertige Code-Sammlungen. Standard Library deckt alles ab. Externe Pakete per pip installieren.

```
# Ganzes Modul laden
import math
math.sqrt(16)   # 4.0
math.pi         # 3.14159...

# Einzelne Funktion laden
from math import sqrt
sqrt(25)        # 5.0

# Mit Alias
import random as r
r.randint(1, 6)  # Würfel 1-6

# Nützliche Module
import os, json, datetime
```

💡 *`pip install paketname` installiert externe Pakete. `pip list` zeigt alle.*